

Holistic Data Preparer — Theory Concepts

Customer Credit Risk: Data Preprocessing & Feature Engineering

Part A — Conceptual Foundation

What is Data Analysis?

Data Analysis is the process of inspecting, cleaning, transforming, and modeling data to discover useful information, draw conclusions, and support decision-making. It spans descriptive analysis (what happened), diagnostic analysis (why it happened), predictive analysis (what will happen), and prescriptive analysis (what should be done about it).

How to Plan a Data Science Project

- **Business understanding** — define the problem and success metric (e.g. reduce loan default losses).
- **Data acquisition** — identify and pull together all relevant sources (CSV, JSON, SQL, APIs).
- **Data understanding** — explore shape, types, distributions, missingness.
- **Data preparation** — cleaning, imputation, outlier handling, encoding, scaling, feature engineering.
- **Modeling** — train and evaluate candidate ML models.
- **Evaluation** — validate against the business metric; check for bias and data leakage.
- **Deployment & monitoring** — ship the model and track drift over time.

In practice, the data understanding and preparation stages typically consume 60-80% of total project time, which is why this project focuses entirely on them.

How to Frame a Machine Learning Problem

Framing an ML problem means translating a business question into a well-posed learning task: choosing the **target variable** (here, `default_flag`), the **problem type** (binary classification), the **unit of prediction** (one row per customer), the **features available before the prediction is needed** (to avoid leakage), and an **evaluation metric** that reflects real business cost — e.g. recall on defaulters or a cost-weighted metric, rather than plain accuracy, since loan defaults are the minority but costly class.

Tensors

A **tensor** is a generalized, n-dimensional array — the core data structure of numerical and deep-learning computing. Its **rank** (or order) is the number of axes it has.

Rank	Name	Example
0	Scalar	a single number, e.g. 42

1	Vector	a list of numbers, e.g. one customer's features
2	Matrix	rows x columns, e.g. many customers x features
3+	Tensor	e.g. a stack/batch of customer matrices

In this project, every pandas DataFrame is backed by NumPy arrays (rank-2 tensors: rows x columns). When features are scaled, encoded, or passed into scikit-learn transformers, they are converted into 2D NumPy tensors — the same structure covered above.

Data Cleaning & Preprocessing — Definitions

Missing Value Imputation

- **Simple Imputer (mean/median):** replaces missing numeric values with the column mean or median.
- **Simple Imputer (most frequent):** replaces missing categorical values with the most common category (the mode).
- **Missing Indicator + Random Sample Imputation:** flags which rows were originally missing (a binary indicator column) and fills the gaps by randomly sampling from the column's own observed distribution, preserving its shape.
- **KNN Imputer:** fills a missing value using the average of the k most similar rows (nearest neighbors) across other numeric columns — a multivariate method.
- **MICE (Multiple Imputation by Chained Equations) / Iterative Imputer:** models each column with missing values as a function of the other columns, iterating until estimates stabilize; more accurate than univariate methods for correlated features.
- **Complete Case Analysis:** simply drops rows or columns that contain missing values; simple but discards information and can bias the sample if data is not missing completely at random.

Outlier Detection & Treatment

- **Z-score method:** flags a value as an outlier if it is more than a chosen number of standard deviations (commonly 3) from the mean.
- **IQR method:** flags values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$, where $IQR = Q3 - Q1$.
- **Percentile method:** flags values outside a chosen low/high percentile band (e.g. below the 1st or above the 99th percentile).
- **Winsorization:** instead of removing outliers, clips (caps) them to a chosen percentile boundary, keeping the row but limiting the extreme value's influence.

Encoding Categorical & Numerical Variables

- **Ordinal Encoding:** maps categories with a natural order (e.g. education level) to integers that preserve that order.
- **Label Encoding:** assigns an arbitrary integer to each category; suitable for binary or tree-based-model-friendly categorical features without inherent order.
- **One-Hot Encoding:** creates a separate binary (0/1) column for each category, avoiding a false sense of order for nominal variables like region or loan purpose.
- **Binning / Discretization:** groups a continuous variable into a smaller number of ranges or bins (e.g. income quartiles).
- **Binarization:** converts a numeric variable into 0/1 based on a threshold (e.g. credit score above 700).
- **Quantile Binning:** splits a variable into bins containing roughly equal numbers of observations.
- **K-Means Binning:** uses the K-Means clustering algorithm on a single variable to define data-driven bin boundaries instead of fixed-width or quantile cuts.

Feature Scaling

- **Standardization (Z-score scaling):** rescales a feature to have mean 0 and standard deviation 1.
- **Normalization / Min-Max Scaling:** rescales a feature to a fixed range, typically [0, 1].
- **MaxAbs Scaling:** divides each value by the maximum absolute value of the feature, preserving sparsity and sign; result falls in [-1, 1].
- **Robust Scaling:** centers and scales a feature using the median and interquartile range instead of the mean/standard deviation, making it far less sensitive to outliers.

Feature Construction & Transformation

- **FunctionTransformer (log, reciprocal, square-root):** applies a custom mathematical function to a column, commonly used to reduce right-skew in financial data.
- **PowerTransformer — Box-Cox:** a parametric power transform (requires strictly positive values) that finds the optimal power to make data more Gaussian-like.
- **PowerTransformer — Yeo-Johnson:** a generalization of Box-Cox that also supports zero and negative values.
- **ColumnTransformer:** a scikit-learn utility that applies different preprocessing steps to different columns (e.g. scaling numeric columns while one-hot encoding categorical columns) within a single, reusable pipeline.
- **Feature Construction:** deriving new, often more predictive, features from existing ones — e.g. Debt-to-Income Ratio = $\text{loan_amount} / \text{annual_income}$.